In [1]:
```
import os
import numpy as np
import polars as pl
import torch
import torch.nn as nn
import torch.optim as optim
import mlflow
import mlflow.pytorch
```

In [2]:
```
# ========================
# CONFIG
# ========================
SENSOR_DISTANCE = 0.5
BATCH_SIZE = 256
EPOCHS = 15
LEARNING_RATE = 1e-3
VAL_SPLIT = 0.2
SEED = 4

MLFLOW_TRACKING_URI = "http://localhost:5000/"
MLFLOW_EXPERIMENT = "akos-da"
MLFLOW_RUN_NAME = None

SENSOR1_FILE = "m1_training.parquet"
SENSOR2_FILE = "m2_training.parquet"
SENSOR1_RESULTS_FILE = "m1_results.parquet"
SENSOR2_RESULTS_FILE = "m2_results.parquet"
AVG_DISTANCE_FILE = "avg_dist.parquet"

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
MAX_CPU_THREADS = os.cpu_count() or 1
torch.set_num_threads(MAX_CPU_THREADS)
torch.set_num_interop_threads(MAX_CPU_THREADS)
if torch.cuda.is_available():
    torch.backends.cudnn.benchmark = True
print(f"Using device: {DEVICE}, torch threads: {MAX_CPU_THREADS}")
```

```
Using device: cuda, torch threads: 16
```

In [3]:
```
# ========================
# DATA LOADING
# ========================
def load_data(file_path):
    if file_path.lower().endswith(".parquet"):
        df = pl.read_parquet(file_path)
    elif file_path.lower().endswith(".csv"):
        df = pl.read_csv(file_path, has_header=False)
    else:
        raise ValueError("Unsupported file format: " + file_path)

    if df.height == 0:
        raise ValueError(f"File {file_path} is empty")

    arr = np.array(df.to_numpy(), dtype=np.float32)
    return torch.tensor(arr, dtype=torch.float32)

sensor1_data = load_data(SENSOR1_FILE)
sensor2_data = load_data(SENSOR2_FILE)
sensor1_results = load_data(SENSOR1_RESULTS_FILE)
sensor2_results = load_data(SENSOR2_RESULTS_FILE)
avg_distance = load_data(AVG_DISTANCE_FILE)

if len(sensor1_data) != len(sensor2_data):
    raise ValueError("Both feature files must have the same number of rows")

if sensor1_data.shape[1] != sensor2_data.shape[1]:
    raise ValueError("Both feature files must have the same number of columns")

if len(sensor1_results) != len(sensor1_data):
    raise ValueError("Prediction target batch 1 must have the same number of rows as the first feature file")

if len(sensor2_results) != len(sensor2_data):
    raise ValueError("Prediction target batch 2 must have the same number of rows as the second feature file")

if len(avg_distance) != len(sensor1_data):
    raise ValueError("Distance labels must have the same number of rows as the feature files")

input_size = sensor1_data.shape[1]
print(f"intput size {input_size}")

print(f"Loaded data: {len(sensor1_data)} samples, {input_size} features each")

dataset_size = len(sensor1_data)
torch.manual_seed(SEED)
indices = torch.randperm(dataset_size)
val_size = int(dataset_size * VAL_SPLIT)
train_size = dataset_size - val_size
val_indices = indices[:val_size]
```

```
train_indices = indices[val_size:]
print(f"Split: {train_size} train, {val_size} val")
```

```
intput size 200
Loaded data: 1219813 samples, 200 features each
Split: 975851 train, 243962 val
```

In [4]:
```
# ========================
# MODEL
# ========================
class Net(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, 120),
            nn.Tanh(),
            nn.Linear(120, 60),
            nn.Tanh(),
            nn.Linear(60, 30),
            nn.Tanh(),
            nn.Linear(30, 15),
            nn.Tanh(),
            nn.Linear(15, 3)
        )

    def forward(self, x):
        return self.net(x)

net = Net(input_size).to(DEVICE)
```

In [5]:
```
# ========================
# LOSS FUNCTION
# ========================
def loss_fn(x1, x2, y1, y2, avg_distance):
    # print(type(x1))
    # for i in range(len(x1)):
    #     print(f"{i} {x1[i]} {y1[i]} {x1[i] + y1[i]}")
    # for i in range(len(x2)):
    #     print(f"{i} {x2[i]} {y2[i]} {x2[i] + y2[i]}")
    y1_actual = x1+y1
    y2_actual = x2+y2
    distance = torch.linalg.norm(y1_actual - y2_actual, dim=1)
    constraint = ((distance - avg_distance) ** 2).mean()

    # small regularization to prevent drift
    reg = 0.01 * ((y1**2).mean() + (y2**2).mean())

    return constraint + reg

# ========================
# TRAINING SETUP
# ========================
optimizer = optim.Adam(net.parameters(), lr=LEARNING_RATE)

# ========================
# TRAINING LOOP
# ========================
if mlflow.active_run():
    mlflow.end_run()
mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
mlflow.set_experiment(MLFLOW_EXPERIMENT)
mlflow.start_run(run_name=MLFLOW_RUN_NAME)
mlflow.log_params({
    "sensor_distance": SENSOR_DISTANCE,
    "batch_size": BATCH_SIZE,
    "epochs": EPOCHS,
    "learning_rate": LEARNING_RATE,
    "val_split": VAL_SPLIT,
    "seed": SEED,
    "input_size": input_size,
})

last_avg_loss = 10
best_loss = float("inf")
best_epoch = -1
for epoch in range(EPOCHS):
    train_perm = train_indices[torch.randperm(train_size)]

    batch_losses = []
    distance_errors = []

    for i in range(0, train_size, BATCH_SIZE):
        indices = train_perm[i:i+BATCH_SIZE]

        batch1 = sensor1_data[indices].to(DEVICE)
        batch2 = sensor2_data[indices].to(DEVICE)
        x1 = sensor1_results[indices].to(DEVICE)
        x2 = sensor2_results[indices].to(DEVICE)
        avg_dist = avg_distance[indices].view(-1).to(DEVICE)
```

```
        y1 = net(batch1)
        y2 = net(batch2)

        loss = loss_fn(x1, x2, y1, y2, avg_dist)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        batch_losses.append(loss.item())

        with torch.no_grad():
            y1_actual = x1 + y1
            y2_actual = x2 + y2
            distance = torch.linalg.norm(y1_actual - y2_actual, dim=1)
            distance_errors.extend((distance - avg_dist).cpu().tolist())

    batch_loss_tensor = torch.tensor(batch_losses, dtype=torch.float32)
    avg_loss = batch_loss_tensor.mean().item()
    std_loss = batch_loss_tensor.std(unbiased=False).item()

    distance_error_tensor = torch.tensor(distance_errors, dtype=torch.float32)
    avg_distance_error = distance_error_tensor.abs().mean().item()
    std_distance_error = distance_error_tensor.std(unbiased=False).item()

    with torch.no_grad():
        val_batch_losses = []
        val_distance_errors = []
        for i in range(0, val_size, BATCH_SIZE):
            indices = val_indices[i:i+BATCH_SIZE]

            batch1 = sensor1_data[indices].to(DEVICE)
            batch2 = sensor2_data[indices].to(DEVICE)
            x1 = sensor1_results[indices].to(DEVICE)
            x2 = sensor2_results[indices].to(DEVICE)
            avg_dist = avg_distance[indices].view(-1).to(DEVICE)

            y1 = net(batch1)
            y2 = net(batch2)

            loss = loss_fn(x1, x2, y1, y2, avg_dist)
            val_batch_losses.append(loss.item())

            y1_actual = x1 + y1
            y2_actual = x2 + y2
            distance = torch.linalg.norm(y1_actual - y2_actual, dim=1)
            val_distance_errors.extend((distance - avg_dist).cpu().tolist())

        val_loss_tensor = torch.tensor(val_batch_losses, dtype=torch.float32)
        val_avg_loss = val_loss_tensor.mean().item() if val_batch_losses else float("nan")
        val_std_loss = val_loss_tensor.std(unbiased=False).item() if val_batch_losses else float("nan")

        val_distance_error_tensor = torch.tensor(val_distance_errors, dtype=torch.float32)
        val_avg_distance_error = val_distance_error_tensor.abs().mean().item() if val_distance_errors else float("nan")
        val_std_distance_error = val_distance_error_tensor.std(unbiased=False).item() if val_distance_errors else float("nan")

    mlflow.log_metrics({
        "train_loss": avg_loss,
        "train_loss_std": std_loss,
        "train_dist_err_mean": avg_distance_error,
        "train_dist_err_std": std_distance_error,
        "val_loss": val_avg_loss,
        "val_loss_std": val_std_loss,
        "val_dist_err_mean": val_avg_distance_error,
        "val_dist_err_std": val_std_distance_error,
    }, step=epoch + 1)

    if val_avg_loss < best_loss:
        best_loss = val_avg_loss
        best_epoch = epoch + 1
        torch.save(net.state_dict(), "best_model.pt")

    if avg_loss > last_avg_loss * 3:
        print(f"Warning: Loss increased significantly from {last_avg_loss:.6f} to {avg_loss:.6f}")
        break
    last_avg_loss = avg_loss
    print(
        f"Epoch {epoch+1}/{EPOCHS}, "
        f"Train Loss: {avg_loss:.6f}, Train Loss std: {std_loss:.6f}, "
        f"Train Dist err mean: {avg_distance_error:.6f}, Train Dist err std: {std_distance_error:.6f}, "
        f"Val Loss: {val_avg_loss:.6f}, Val Loss std: {val_std_loss:.6f}, "
        f"Val Dist err mean: {val_avg_distance_error:.6f}, Val Dist err std: {val_std_distance_error:.6f}"
    )

mlflow.end_run()
```

```
Epoch 1/15, Train Loss: 0.013313, Train Loss std: 0.027061, Train Dist err mean: 0.015885, Train Dist err std: 0.113720, Val Loss: 0.045077, Val Loss std: 0.043436, Val Dist err mean: 0.025119, Val Dist err std: 0.211953
Epoch 2/15, Train Loss: 0.016750, Train Loss std: 0.030857, Train Dist err mean: 0.017388, Train Dist err std: 0.128285, Val Loss: 0.002300, Val Loss std: 0.001715, Val Dist err mean: 0.011264, Val Dist err std: 0.045717
Epoch 3/15, Train Loss: 0.001916, Train Loss std: 0.003609, Train Dist err mean: 0.010352, Train Dist err std: 0.040781, Val Loss: 0.002842, Val Loss std: 0.002677, Val Dist err mean: 0.010767, Val Dist err std: 0.050210
Epoch 4/15, Train Loss: 0.001656, Train Loss std: 0.003152, Train Dist err mean: 0.009661, Train Dist err std: 0.037330, Val Loss: 0.001266, Val Loss std: 0.000697, Val Dist err mean: 0.009464, Val Dist err std: 0.032350
Epoch 5/15, Train Loss: 0.001328, Train Loss std: 0.002140, Train Dist err mean: 0.009018, Train Dist err std: 0.033292, Val Loss: 0.001044, Val Loss std: 0.000626, Val Dist err mean: 0.008407, Val Dist err std: 0.029045
Epoch 6/15, Train Loss: 0.001319, Train Loss std: 0.001255, Train Dist err mean: 0.009009, Train Dist err std: 0.033014, Val Loss: 0.001110, Val Loss std: 0.000644, Val Dist err mean: 0.008553, Val Dist err std: 0.030116
Epoch 7/15, Train Loss: 0.002968, Train Loss std: 0.009190, Train Dist err mean: 0.010814, Train Dist err std: 0.051909, Val Loss: 0.001059, Val Loss std: 0.000605, Val Dist err mean: 0.008818, Val Dist err std: 0.029290
Epoch 8/15, Train Loss: 0.001343, Train Loss std: 0.001646, Train Dist err mean: 0.009141, Train Dist err std: 0.033408, Val Loss: 0.001160, Val Loss std: 0.000672, Val Dist err mean: 0.008851, Val Dist err std: 0.030997
Epoch 9/15, Train Loss: 0.002253, Train Loss std: 0.007135, Train Dist err mean: 0.009581, Train Dist err std: 0.044761, Val Loss: 0.001219, Val Loss std: 0.002017, Val Dist err mean: 0.009174, Val Dist err std: 0.031616
Epoch 10/15, Train Loss: 0.001590, Train Loss std: 0.002861, Train Dist err mean: 0.009366, Train Dist err std: 0.036497, Val Loss: 0.001110, Val Loss std: 0.000657, Val Dist err mean: 0.008834, Val Dist err std: 0.030377
Epoch 11/15, Train Loss: 0.001213, Train Loss std: 0.001241, Train Dist err mean: 0.008702, Train Dist err std: 0.031530, Val Loss: 0.001262, Val Loss std: 0.000707, Val Dist err mean: 0.009301, Val Dist err std: 0.031860
Epoch 12/15, Train Loss: 0.001222, Train Loss std: 0.001067, Train Dist err mean: 0.008805, Train Dist err std: 0.031697, Val Loss: 0.001194, Val Loss std: 0.000684, Val Dist err mean: 0.008788, Val Dist err std: 0.031447
Epoch 13/15, Train Loss: 0.001176, Train Loss std: 0.001130, Train Dist err mean: 0.008580, Train Dist err std: 0.031007, Val Loss: 0.002491, Val Loss std: 0.001452, Val Dist err mean: 0.011193, Val Dist err std: 0.044371
Epoch 14/15, Train Loss: 0.001175, Train Loss std: 0.001166, Train Dist err mean: 0.008586, Train Dist err std: 0.030961, Val Loss: 0.001085, Val Loss std: 0.000637, Val Dist err mean: 0.008710, Val Dist err std: 0.029601
Epoch 15/15, Train Loss: 0.001271, Train Loss std: 0.001015, Train Dist err mean: 0.008900, Train Dist err std: 0.032474, Val Loss: 0.001061, Val Loss std: 0.000639, Val Dist err mean: 0.008404, Val Dist err std: 0.029525
🏃 View run likeable-bat-14 at: http://localhost:5000/#/experiments/1/runs/737829a7390f41f6baf2a90ac8aab869
🧪 View experiment at: http://localhost:5000/#/experiments/1
```

In [6]:

```python
# ==========================
# SAVE MODEL
# ==========================
torch.save(net.state_dict(), "model.pt")

# Export only the best model to ONNX.
best_path = "best_model.pt"
best_onnx = "best_model.onnx"
if os.path.exists(best_path):
    net.load_state_dict(torch.load(best_path, map_location=DEVICE))
    net.eval()
else:
    print("Warning: best_model.pt not found; exporting current model.")

dummy_input = torch.randn(1, input_size, device=DEVICE)
torch.onnx.export(
    net,
    dummy_input,
    best_onnx,
    export_params=True,
    opset_version=17,
    do_constant_folding=True,
    dynamo=False,
    input_names=["input"],
    output_names=["output"],
    dynamic_axes={"input": {0: "batch_size"}, "output": {0: "batch_size"}},
)

print(
    f"Exported {best_onnx} from best_model.pt (epoch {best_epoch})."
)
```

```
Exported best_model.onnx from best_model.pt (epoch 5).
/tmp/nix-shell.eS6OiP/nix-shell.b8l2GD/ipykernel_76880/538843678.py:16: DeprecationWarning: You are using the legacy TorchScript-based ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutorials/beginner/onnx/export_control_flow_model_to_onnx_tutorial.html
  torch.onnx.export(
```